

0-k BFS

Bounded Integer Weights Shortest Path – C++17

Artemov Makar Mark Shkut

Skoltech, Algorithms 2026

May 21, 2026

Open with the headline.

Project: 0-k BFS for the bounded-integer-weights shortest path problem in C++17. Joint work with Mark. Goal of this presentation: explain the algorithm family, show why it beats Dijkstra on graphs with small integer weights, and present our measurements up to $V = 9 \cdot 10^8$. Keep it tight: about 15 minutes plus questions.

The problem

- Single-source shortest path on $G = (V, E, w)$
- Weights restricted to $\{0, 1, \dots, k\}$ for small known k
- Target: V approaching 10^9 on a single workstation

Where this comes up: grid path planning, layered graphs, 2-SAT-style reductions, transport networks with a few toll levels.

Define the problem precisely.

We want all distances from one source. The catch is that the weights are non-negative integers bounded by a small k . This restriction is not artificial: it shows up constantly in pathfinding on grids, in layered DAGs, and in competitive programming. The reason it is interesting from an algorithms standpoint is that we can do better than the classical $O((V + E) \log V)$ Dijkstra by exploiting the small range of weights.

Why the $V \rightarrow 10^9$ target: we set this as a forcing function on the design – it dictates the implicit-graph backend and the 32-bit distance type that come later.

- `std::priority_queue<pair<Distance,Vertex>>`, lazy decrease-key
- $O((V + E) \log V)$ time, $O(V + E)$ memory
- Works for any non-negative real weights

The $\log V$ factor is the cost of the priority queue. *If we know weights are bounded, can we replace it with something cheaper?*

Set up Dijkstra as the benchmark to beat.

This is the standard algorithm everyone knows. We implement it straightforwardly: a heap of (distance, vertex) pairs with lazy decrease-key (re-push, skip stale on pop). The $\log V$ factor sits inside every relaxation that improves a distance. The whole point of 0-k BFS is that when the weights are small integers, you do not need a heap at all.

Pause for the question on the slide – this motivates the next slide.

The trick: 0–1 BFS

Weights are 0 or 1. Replace the priority queue with a **deque**:

- edge of weight 0 $\rightarrow$ push_front
- edge of weight 1 $\rightarrow$ push_back

The deque is always sorted by distance, in two contiguous layers.

Complexity: $O(V + E)$ **Memory:** $O(V)$

Pedagogical core, part 1.

The idea is simple. Inside any BFS we always have “current layer” and “next layer”. With 0/1 weights, a 0-edge keeps us in the current layer (front of the deque, processed immediately) and a 1-edge moves us to the next layer (back of the deque). The deque stays sorted by distance with at most two distinct values at any time.

Important detail vs. plain BFS: a vertex can be discovered multiple times before it is final, so we must check $dist[v] + w(v,u) < dist[u]$ on every relaxation, not just “visited or not”.

Weights are $\{0, 1, \dots, k\}$. Use $k + 1$ **FIFO queues**, indexed by $\text{dist}[v] \bmod (k + 1)$.

- “Active” bucket cur walks forward
- Relaxation pushes v into bucket $(d_{\text{new}}) \bmod (k + 1)$
- Stale entries are skipped on pop

Complexity: $O(kV + E)$ **Memory:** $O(k + E)$ for queues

Pedagogical core, part 2.

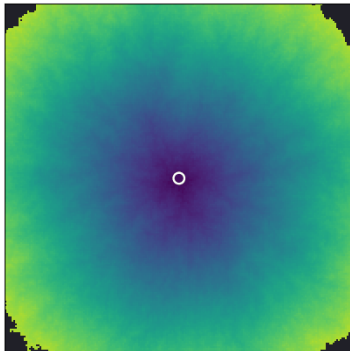
Generalize: with weights up to k , a vertex relaxed at distance d moves at most $d + k$ steps forward. So at any moment, only $k + 1$ “layers” of distance values are alive. We allocate $k + 1$ FIFO queues and index them by $\text{dist} \bmod (k + 1)$ – a circular buffer of buckets.

This is why it is called “circular FIFO queues”. The `cur` pointer walks through them; when the current bucket empties, we advance to the next. If a vertex is re-relaxed to a smaller distance, it lands in a different bucket, and the original slot becomes a stale entry that we skip on pop.

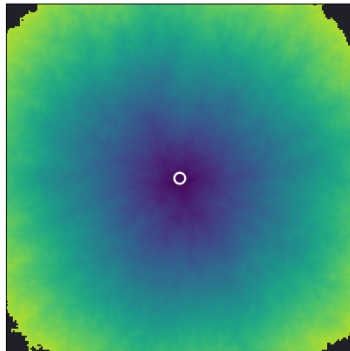
What it looks like

Same grid (200x200, $k=8$) · same answer · Dijkstra is 3.32x slower

0-k BFS · 4.00 ms



Dijkstra · 13.30 ms



$d \leq 300.0$ reached 39273 / 40000 (98.2%)

Same 200×200 grid, $k = 8$. Same answer. 0-k BFS **4.00 ms**, Dijkstra **13.30 ms** – 3.32× slower.

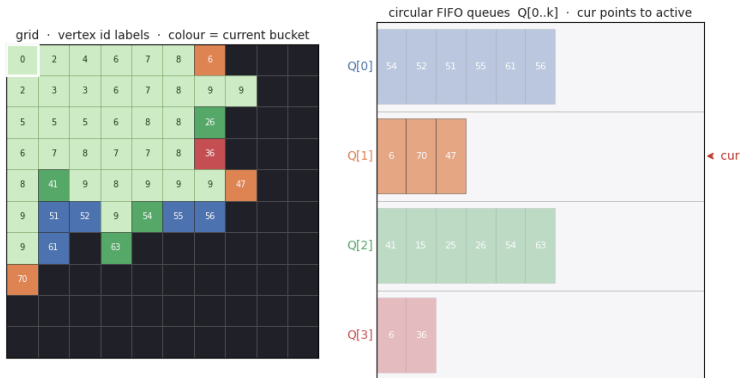
Show the comparison animation (or this still).

“Both algorithms produce the same distance field. The left panel finishes the wavefront in 4 milliseconds, the right one in 13.3 milliseconds because every relaxation pays the log-factor in the heap. The animation in the repository runs this live – you can see both wavefronts filling in lockstep with the real times in the titles.”

Talking point: the visualization is also part of the project – it is how we communicated the work to ourselves before benchmarking.

Under the hood

0-k BFS step-by-step · grid 10x10, k=3



step 37: pop $u=17$ from $Q[1]$ (cur=9)

Step-by-step on a tiny 10×10 grid, $k = 3$. Each row on the right is one queue $Q[0] \dots Q[3]$; **cur** marks the active bucket.

Anchor the data structure.

On the right you see the actual four queues $Q[0]$ through $Q[3]$ with their current contents (the numbers are vertex ids). On the left you see the grid, with cells coloured by which bucket currently owns them. The `cur` pointer is on $Q[1]$ in this frame.

The animation is in `viz/animate_queues.py` and it consumes a per-step trace dumped by the instrumented C++ algorithm (`include/zkbfs/bfs0k_trace.hpp`). This is the part that makes the algorithm *teachable* – the queues are the whole secret.

- Explicit CSR at $V = 10^9$ costs ~ 20 GB just for edges – *infeasible*
- GridGraph: implicit 4-neighbour grid, $O(1)$ neighbour function, no edge storage
- Distance = uint32_t: halves the dominant array (4 GB instead of 8 GB)
- Trade-off only valid while $k \cdot V < 2^{32}$

Defend the design decisions that made the big run possible.

We split the project into two backends. The general GraphCSR caps out around $V \approx 10^8$ because of memory. The GridGraph is implicit – we do not store edges at all, we compute them on the fly. That, plus a 32-bit distance type, is what let us actually run on $V = 9 \cdot 10^8$ within 14 GB of free RAM.

Honest disclaimer: the 32-bit distance is not always safe. We require $k \cdot V < 2^{32}$. For the parameters we care about, this is fine, and we have a 64-bit fallback via a compile flag.

Results, grid, single thread

V	k	Dijkstra	0- k BFS	0-1 BFS	speed-up
10^6	1	0.180 s	0.039 s	0.070 s	4.6×
10^7	1	1.98 s	0.493 s	0.355 s	5.6×
10^8	1	60.3 s	28.4 s	5.44 s	11.1 ×
$4.84 \cdot 10^8$	4	432.9 s	451.4 s	—	0.96× (mem)
$9 \cdot 10^8$	1	619.9 s	—	121.2 s	5.12 ×

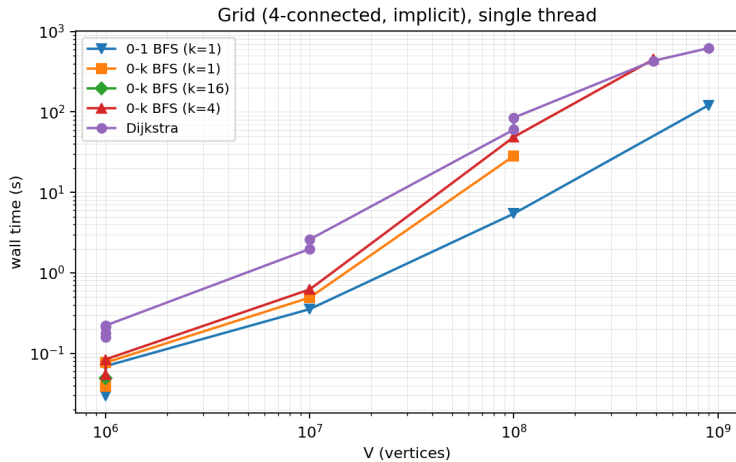
Both algorithms produce *byte-identical* distance checksums on every graph instance.

Walk the table top to bottom.

At $V = 10^6$ we already get a $4\text{--}5\times$ speed-up. At $V = 10^8$ with $k = 1$, the deque-based 0–1 BFS is $11\times$ faster than Dijkstra. The $V \approx 5 \cdot 10^8$ row is interesting – the gap collapses because both algorithms become memory-bandwidth bound. The bottom row is the headline: on 900 million vertices, 0–1 BFS finishes in two minutes, Dijkstra takes ten.

Critically, all checksums match – correctness was verified across 19 distinct graph instances in the sweep, zero mismatches.

Scaling



Walk the plot.

Both axes are log scale. Five series: Dijkstra at the top, the BFS variants tracking parallel below it. The constant vertical gap on the log scale corresponds to a roughly constant multiplicative speed-up. At $V = 10^8$ and beyond, the lines start to converge as we move into the memory-bound regime. The lowest line throughout is the deque-based 0–1 BFS, which is the cleanest implementation in the family.

- Speed-up shrinks as k grows – gone at $k = \omega(\log V)$
- At $V \gtrsim 5 \cdot 10^8$ both algorithms become memory-bandwidth bound
- Two-queue *bfs_0k* is slower than single-deque *bfs_01* at large V (allocator overhead) – $k + 1$ queues = $k + 1$ allocators
- Needs non-negative integer weights with known small k

Be honest about where this fails.

First: this is a constant-factor win, not an asymptotic one. Once k is proportional to $\log V$, Dijkstra is at least as good.

Second: at huge V everything is memory-bound. We measured the 0- k BFS slowing down to parity with Dijkstra at $V \approx 5 \cdot 10^8$. Implementation matters a lot: maintaining $k + 1$ separate queue allocators hurts cache and allocator pressure compared to one deque, which is why our 0-1 BFS keeps winning at $V = 9 \cdot 10^8$ but the generic 0- k BFS would not.

Third: this is not a replacement for Dijkstra in general. Real-valued weights or unknown weight bounds – use Dijkstra.

- Indexed d -ary heap Dijkstra (proper decrease-key)
- Chunked ring-buffer FIFO for `bfs_0k` (close the gap to 0–1 BFS)
- Real road-network loader (DIMACS USA, weights quantised to $k = 8$)
- Stretch: Δ -stepping for parallel context

Forward look.

Four items. The first two are implementation polish – a real decrease-key heap to be a fairer baseline, and a chunked queue to match the deque's cache behaviour. Three: leave the synthetic-graph comfort zone and try a real road network. Four: Δ -stepping is the canonical parallel relative of 0-k BFS; it would be nice to have for context even if we do not push the parallel side hard.

Thank you

- Code, report, plots: `github.com/artemovmak/EffectiveAlgorithms`
- Stage 1 report: `stage1_report.pdf`

Questions?

Close.

Everything – the code, the LaTeX report, the animations, reproducible build scripts – lives in the GitHub repo. Hand off to questions. Likely questions:

Q: “Why is 0–1 BFS faster than 0– k BFS at $k = 1$?” – allocator overhead of two `std::queues` vs a single `std::deque`.

Q: “How did you verify correctness on a billion-vertex run?” – distance-sum checksums match Dijkstra exactly; no per-vertex comparison needed because the sum already pins down the array.

Q: “What about parallelism?” – single thread throughout Stage 1; Δ -stepping is the planned Week 2 stretch goal.